

Lecture03: Exploratory data analysis

Yomin E. Jaramillo Múnera
EAFIT



Contenido

1. Adquisición y limpieza de datos
2. EDA
3. Transformación de columnas



Knowledge refresh

Actividad

Cargar o crear un dataset para aplicar al menos 5 funcionalidades de Pandas que a ustedes les parezca interesante o no hayan probado.

Posibles fuentes de datasets:

- <https://www.kaggle.com/datasets>
- <https://ieee-dataport.org/datasets>
- <https://mlcontests.com/>



Adquisición y limpieza de datos

Con un dataset limpio y bien estructurado el performance de los modelos puede aumentar

Dataset “sucio”

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# 1. Generar datos base
np.random.seed(41)
n_samples = 500

# Variables: Edad y Puntuación (0-100)
edad = np.random.randint(18, 70, n_samples)
puntuacion = np.random.randint(1, 100, n_samples)

# Crear el target (Compra: 1 si es joven y gasta mucho, o si es mayor y gasta poco)
# Añadimos ruido para que el accuracy no sea perfecto
target = ((edad < 40) & (puntuacion > 50) | (edad > 40) & (puntuacion < 30)).astype(int)
ruido = np.random.choice([0, 1], size=n_samples, p=[0.85, 0.15])
target = np.logical_xor(target, ruido).astype(int)

df = pd.DataFrame({'Edad': edad, 'Puntuacion': puntuacion, 'Compro': target})

# --- ENSUCIAR EL DATASET ---

# A. Insertar valores nulos (NaN)
df.loc[df.sample(frac=0.1).index, 'Edad'] = np.nan

# B. Insertar Outliers (Valores imposibles)
df.loc[0:5, 'Edad'] = 250 # Alguien de 250 años
df.loc[6:10, 'Puntuacion'] = -50 # Puntuación negativa

# C. Insertar Duplicados
df = pd.concat([df, df.iloc[:20]], ignore_index=True)

# D. Columna categórica con errores de escritura
países = ['España', 'Mexico', 'Colombia', 'Argentina']
df['Pais'] = np.random.choice(países, len(df))
df.loc[df.sample(frac=0.05).index, 'Pais'] = 'Españaa' # Error tipográfico

print("Dataset sucio creado. Primeras filas:")
print(df.head())
```

Resultado algoritmo ML

```
#Algoritmo de ML
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# 1. Limpieza básica para que el modelo pueda ejecutarse
# Rellenamos los nulos de 'Edad' con la mediana (para evitar que el outlier de 250 afecte tanto)
df_limpio = df.copy()
df_limpio['Edad'] = df_limpio['Edad'].fillna(df_limpio['Edad'].median())

# Convertimos la columna 'Pais' a números (Label Encoding rápido)
df_limpio['Pais'] = df_limpio['Pais'].astype('category').cat.codes

# 2. División de datos (Features X y Target y)
X = df_limpio.drop('Compro', axis=1)
y = df_limpio['Compro']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Entrenamiento del Modelo
modelo = RandomForestClassifier(n_estimators=100, random_state=42)
modelo.fit(X_train, y_train)

# 4. Predicción y Evaluación
y_pred = modelo.predict(X_test)

print(f"Accuracy final: {accuracy_score(y_test, y_pred):.2f}")
print("\nMatriz de Confusión:")
print(confusion_matrix(y_test, y_pred))
```

Accuracy final: 0.78



Adquisición y limpieza de datos

Dataset limpio

```
# 1. Separar las clases
df_mayoritaria = df_final[df_final.Compro == 0]
df_minoritaria = df_final[df_final.Compro == 1]

# 2. Sobremuestrear la clase minoritaria
df_minoritaria_upsampled = resample(df_minoritaria,
                                    replace=True,      # duplicar muestras
                                    n_samples=len(df_mayoritaria), # igualar a la mayoritaria
                                    random_state=42)

# 3. Combinar de nuevo
df_balanceado = pd.concat([df_mayoritaria, df_minoritaria_upsampled])

# 4. Re-entrenar
X_bal = df_balanceado.drop(['Compro', 'Pais'], axis=1)
y_bal = df_balanceado['Compro']
```

Resultado algoritmo ML

```
X_train_b, X_test_b, y_train_b, y_test_b = train_test_split(X_bal, y_bal, test_size=0.2, random_state=42)

modelo_final = RandomForestClassifier(n_estimators=100, random_state=42)
modelo_final.fit(X_train_b, y_train_b)

# 5. Evaluación final
y_pred_final = modelo_final.predict(X_test_b)
print(f"Accuracy con datos limpios y balanceados: {accuracy_score(y_test_b, y_pred_final):.2f}")
print("\nReporte de Clasificación:")
print(classification_report(y_test_b, y_pred_final))
```

Accuracy con datos limpios y balanceados: 0.82



Adquisición y limpieza de datos

TIPOS DE DATOS

Estructurados:

Poseen formato predefinido y una organización clara y consistente.

Altamente organizados, lo que permite una fácil manipulación y búsqueda.

Menos flexible: Su estructura predefinida puede limitar su uso para propósitos distintos a los originales.

Definición previa: se debe definir la estructura de los datos (columnas, tipos de datos, relaciones) antes de poder almacenar cualquier información.

Dificultad de cambio: Alterar este esquema (añadir una nueva columna, cambiar un tipo de dato) después de que los datos ya están almacenados puede ser una operación costosa y compleja, especialmente en grandes volúmenes de datos.



Adquisición y limpieza de datos

TIPOS DE DATOS

Estructurados:

Bases de datos relacionales (SQL): MySQL, PostgreSQL, Oracle, SQL Server. Contienen información como registros de clientes, transacciones financieras, inventarios, etc.

Hojas de cálculo: Archivos CSV, Archivos de Excel o Google Sheets con datos organizados en celdas



Adquisición y limpieza de datos

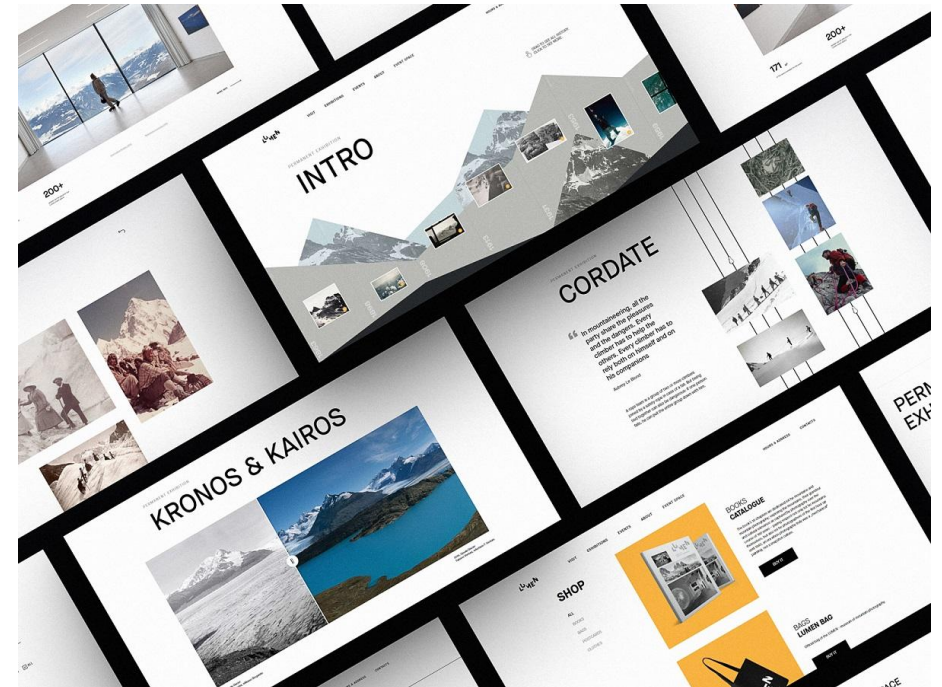
TIPOS DE DATOS

Estructurados:

Fuentes de datos estructurados:

Formularios web: Resultados de encuestas o registros en línea con campos definidos.

Sensores y dispositivos IoT: Datos de geolocalización, temperatura, presión, etc., cuando se presentan en un formato tabular.



Adquisición y limpieza de datos

TIPOS DE DATOS

Semiestructurados:

Tienen cierta estructura, pero no siguen un esquema fijo y rígido como los datos estructurados.

Utilizan etiquetas o marcadores para organizar y separar elementos de datos, pero no necesariamente se organizan en tablas.

- Archivos XML (Extensible Markup Language): Utilizan etiquetas para definir y organizar los datos.
- Archivos JSON (JavaScript Object Notation): Formato ligero para el intercambio de datos, muy común en APIs web.



Adquisición y limpieza de datos

TIPOS DE DATOS

Semiestructurados:

Tienen cierta estructura, pero no siguen un esquema fijo y rígido como los datos estructurados.

Utilizan etiquetas o marcadores para organizar y separar elementos de datos, pero no necesariamente se organizan en tablas.

- Archivos XML (Extensible Markup Language): Utilizan etiquetas para definir y organizar los datos.
- Archivos JSON (JavaScript Object Notation): Formato ligero para el intercambio de datos, muy común en APIs web.



Adquisición y limpieza de datos

TIPOS DE DATOS

No estructurados:

No poseen formato predefinido ni una estructura organizacional clara.

No se ajustan a un modelo de datos tradicional y pueden ser difíciles de categorizar y analizar.

Son abundantes y se generan de forma continua (se estima que representan el 80-90% de los datos).

Flexibles y escalables: Al no tener una estructura rígida, son más adaptables a nuevas necesidades de análisis.



Adquisición y limpieza de datos

TIPOS DE DATOS

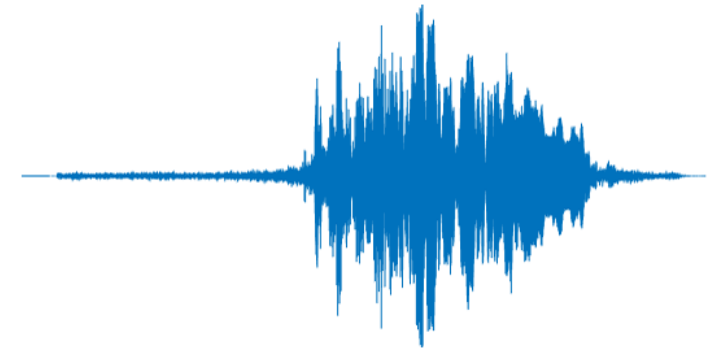
No estructurados:

Texto: Documentos (PDF, Word), correos electrónicos, publicaciones en redes sociales, comentarios, reseñas, artículos, libros.

Multimedia: Imágenes (JPG, PNG), videos (MP4, AVI), audio (MP3, WAV).

Datos de comunicación: Mensajes de chat, grabaciones de llamadas.

Datos científicos: Informes sísmicos, datos de exploración espacial, resonancias magnéticas, radiografías.



Adquisición y limpieza de datos

CARGA DE ARCHIVOS

CSVs

```
import pandas as pd  
  
brics = pd.read_csv("/content/drive/My Drive/Colab Notebooks/fundCD202520/brics.csv", index_col = 0)  
  
brics
```

	country	capital	area	population
BR	Brazil	Brasilia	8.516	200.40
RU	Russia	Moscow	17.100	143.50
IN	India	New Delhi	3.286	1252.00
CH	China	Beijing	9.597	1357.00
SA	South Africa	Pretoria	1.221	52.98



Adquisición y limpieza de datos

CARGA DE ARCHIVOS

CSVs

```
import pandas as pd

clientes = pd.read_csv("/content/drive/My Drive/Colab Notebooks/fundCD202520/brics_spaces.csv", sep='\s+')

clientes
```

brics.csv		brics_spaces.csv x	
Nombre	Edad	Ciudad	Ocupacion
Alice	30	"Nueva York"	Ingeniera
Bob	24	Londres	Diseñador
Charlie	35	París	"Científico de Datos"

	Nombre	Edad	Ciudad	Ocupacion
0	Alice	30	Nueva York	Ingeniera
1	Bob	24	Londres	Diseñador
2	Charlie	35	París	Científico de Datos



Adquisición y limpieza de datos

CARGA DE ARCHIVOS

Excel

```
import pandas as pd

clientes = pd.read_excel("/content/drive/My Drive/Colab Notebooks/fundCD202520/clientes.xlsx")

clientes
```

	A	B
1	Nombre	Edad
2	Alice	30
3	Bob	24

	Nombre	Edad
0	Alice	30
1	Bob	24



Adquisición y limpieza de datos

APROXIMACIÓN INICIAL

```
>>> # Ver las primeras 5 filas
>>> df.head()

>>> # Ver las ultimas 5 filas
>>> df.tail()

# Dimensiones (filas, columnas)
>>> df.shape

# Tipos de datos y memoria usada
>>> df.info()

# Ver los tipos de datos de cada columna.
>>> df.dtypes
```

```
# Estadísticas descriptivas (media, min, max, percentiles)
>>> df.describe()

# Conteo de valores unicos por columna
>>> df.nunique()

# Conteo de valores unicos en una columna especifica
>>> df["columna"].nunique()

# Elementos unicos por columna
>>> df["columna"].unique()
```



Adquisición y limpieza de datos

Manejo de valores nulos

```
# Detectar nulos por columna (suma total)
>>> df.isnull().sum()

# Total de valores nulos en todo el DataFrame
>>> df.isnull().sum().sum()

# Porcentaje de NaN por columna
>>> df.isnull().sum() / len(df) * 100

# Eliminar filas que tengan AL MENOS un nulo
>>> df_limpio = df.dropna()

# Eliminar columnas que tengan AL MENOS un nulo
>>> df_limpio = df.dropna(axis=1)

# Deja filas que tienen N cantidad de valores no nulos en adelante.
>>> df_limpio = df.dropna(thresh=N)

# Eliminar fila donde TODAS los valores sean nulos.
>>> df_limpio = df.dropna(how='all')
```

```
# Rellenar nulos con un valor constante.
>>> df.fillna(0, inplace=True)

# Rellenar nulos con un valor constante en una columna especifica
>>> df.fillna({"area": 0}, inplace=True)

# Imputacion inteligente: usar la media
>>> df.fillna({"area": df["area"].mean()}, inplace=True)

# Imputacion inteligente: usar la mediana
>>> df.fillna({"area": df["area"].median()}, inplace=True)

# Imputacion inteligente: usar la moda
>>> df.fillna({"area": df["area"].mode()[0]}, inplace=True)

# Rellena con el valor anterior.
>>> df["area"] = df["area"].ffill()

# Rellena con el valor posterior.
>>> df["area"] = df["area"].bfill()
```

```
# Rellena con interpolacion.
>>> df['sueldo'] = df['sueldo'].interpolate()

# Rellenar NaN en la columna 'Nombre' con 'Desconocido'
>>> df['Nombre'].fillna('Desconocido', inplace=True)
```

$$y = y_1 + \frac{x - x_1}{x_2 - x_1}(y_2 - y_1)$$



Adquisición y limpieza de datos

MANIPULACIÓN ROWS/COLUMNS

```
# Eliminar columnas específicas
>>> df.drop(['ID', 'Direccion'], axis=1, inplace=True)

# Eliminar filas por índice
>>> df.drop([0, 1, 2], axis=0)

# Renombrar columnas
>>> df.rename(columns={'old_name': 'new_name'}, inplace=True)

# Renombrar todas las columnas.
>>> df.columns = ['nueva_col1', 'nueva_col2', ...]

# Reordenar columnas.
>>> df = df[['col_ordenada1', 'col_ordenada2', ...]]

# Crear una nueva columna basada en otras
>>> df['total'] = df['precio'] * df['cantidad']
```

```
# Reemplazar valores de texto
>>> df['pais'] = df['pais'].replace('Colombiaa', 'Colombia')

# Cuenta el número total de filas duplicadas.
>>> df.duplicated().sum()

# Eliminar filas duplicadas
>>> df.drop_duplicates(inplace=True)

# Elimina duplicados basándose en un subconjunto de columnas
manteniendo la última ocurrencia.
>>> df = df.drop_duplicates(subset=['area', 'population'])

# Devuelve una Serie booleana indicando filas duplicadas (la
primera ocurrencia no se marca).
>>> df.duplicated()

# Busca duplicados solo en un subconjunto de columnas.
>>> df.duplicated(subset=['col1', 'col2']).sum()

# Convertir a minúsculas y quitar espacios en blanco
>>> df['ciudad'] = df['ciudad'].str.lower().str.strip()
```

Adquisición y limpieza de datos

Actividad

Descargar alguno de estos dos datasets:

- <https://www.kaggle.com/datasets/bharatnatrayn/movies-dataset-for-feature-extracion-prediction>
- <https://www.kaggle.com/datasets/yasserh/titanic-dataset>

Limpiar lo máximo posible el dataset.



Antes de aplicar modelos complejos, un científico de datos debe entender la naturaleza de sus datos.

La estadística descriptiva permite:

Resumir y visualizar: Calcular medidas de tendencia central (media, mediana, moda), medidas de dispersión (varianza, desviación estándar, rango) y visualizaciones efectivas para tener una visión rápida y concisa de los datos.

Identificar patrones y anomalías: Graficar distribuciones, diagramas de dispersión, histogramas para visualizar la distribución de los datos, identificar valores atípicos (outliers) y patrones iniciales.



Fuente: <https://emeritus.org/in/learn/what-is-artificial-intelligence/>



Herramientas y medidas para resumir y presentar los datos:

Medidas de Tendencia Central: Indican el "centro" o el valor típico de un conjunto de datos.

Mean: La suma de todos los valores dividida por el número total de valores. Es sensible a los valores atípicos (outliers).

Median: El valor central en un conjunto de datos ordenado. Si hay un número par de datos, es el promedio de los dos valores centrales. Es robusta a los outliers.

Moda: El valor que aparece con mayor frecuencia en un conjunto de datos. Útil para datos categóricos.



EDA

Medidas de Dispersión (o Variabilidad):

Indican qué tan dispersos o variados están los datos alrededor del centro

Rango: La diferencia entre el valor máximo y el valor mínimo del conjunto de datos.

```
rango_matematicas = df['Matematicas'].max() -  
df['Matematicas'].min()
```

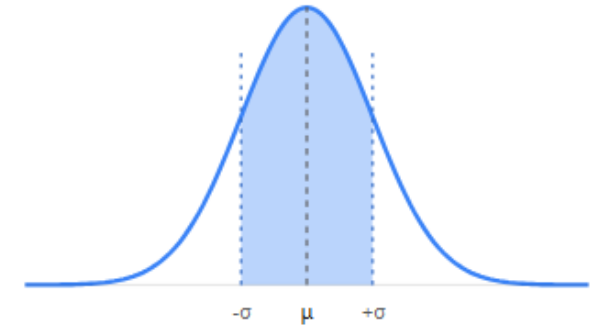
Varianza: Mide la dispersión promedio de los puntos de datos con respecto a la media. Se calcula como el promedio de los cuadrados de las diferencias de cada valor con respecto a la media.

```
varianza_matematicas = df['Matematicas'].var()
```

- **Desviación Estándar:** La raíz cuadrada de la varianza. Es la medida de dispersión más común y se interpreta en las mismas unidades que los datos originales.

```
std_matematicas = df['Matematicas'].std()
```

Var(X) $\approx$ 1.96



Medidas de Posición: Dividen el conjunto de datos en partes iguales.

Cuartiles: dividen un conjunto de datos ordenado en cuatro partes iguales, o "cuartos", cada uno conteniendo aproximadamente el 25% de las observaciones.

Q1 (Primer Cuartil o Percentil 25): Es el valor por debajo del cual cae el 25% más bajo de los datos.

Q2 (Segundo Cuartil o Percentil 50): Es el valor por debajo del cual cae el 50% más bajo de los datos (o el 50% más alto de los datos cae por encima de él).

Q3 (Tercer Cuartil o Percentil 75): Es el valor por debajo del cual cae el 75% más bajo de los datos (o el 25% más alto de los datos cae por encima de él).

Q1 (25%): 74.00
Q2 (50%): 80.00
Q3 (75%): 86.00
IQR: 12.00

Calificacion	
0	40
1	65
2	68
3	70
4	72
5	73
6	74
7	75
8	76
9	77
10	78
11	79
12	80
13	81
14	82
15	83
16	84
17	85
18	86
19	87
20	88
21	89
22	90
23	92
24	95

```
# --- Calcular Q1, Q3 y el IQR para identificar outliers ---  
Q1 = df_calificaciones['Calificacion'].quantile(0.25)  
Q2 = df_calificaciones['Calificacion'].quantile(0.5)  
Q3 = df_calificaciones['Calificacion'].quantile(0.75)  
  
IQR = Q3 - Q1
```

IQR (Rango Intercuartílico): representa el rango del 50% central de un conjunto de datos.



```
lower_bound_outlier = Q1 - 1.5 * IQR #outlier por debajo.
upper_bound_outlier = Q3 + 1.5 * IQR #outlier por encima.

print(f"\nQ1 (25%): {Q1:.2f}")
print(f"Q2 (50%): {Q2:.2f}")
print(f"Q3 (75%): {Q3:.2f}")
print(f"IQR: {IQR:.2f}")
print(f"Límite inferior para outliers: {lower_bound_outlier:.2f}")
print(f"Límite superior para outliers: {upper_bound_outlier:.2f}")

# Identificar outliers
outliers = df_calificaciones[(df_calificaciones['Calificacion'] < lower_bound_outlier)
                             (df_calificaciones['Calificacion'] > upper_bound_outlier)]

print("\nValores considerados outliers según la regla del 1.5 * IQR:")
print(outliers)
```

Límite inferior para outliers: 56.00
Límite superior para outliers: 104.00

Valores considerados outliers según la regla del 1.5 * IQR:

Calificacion
0
40

#outlier por debajo.

$$\begin{aligned} \text{lower_bound_outlier} &= Q1 - 1.5 * IQR \\ &= 74 - 1.5 * 12 \\ &= 74 - 18 \\ &= 56 \end{aligned}$$

#outlier por encima.

$$\begin{aligned} \text{upper_bound_outlier} &= Q1 + 1.5 * IQR \\ &= 74 + 1.5 * 12 \\ &= 74 + 18 \\ &= 86 \end{aligned}$$




```
calificaciones_datos = [
    40, 65, 68, 70, 72, 73, 74,
    75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85,
    86, 87, 88, 89, 90, 92, 95
]
```

Q1 (25%): 74.00

Q2 (50%): 80.00

Q3 (75%): 86.00

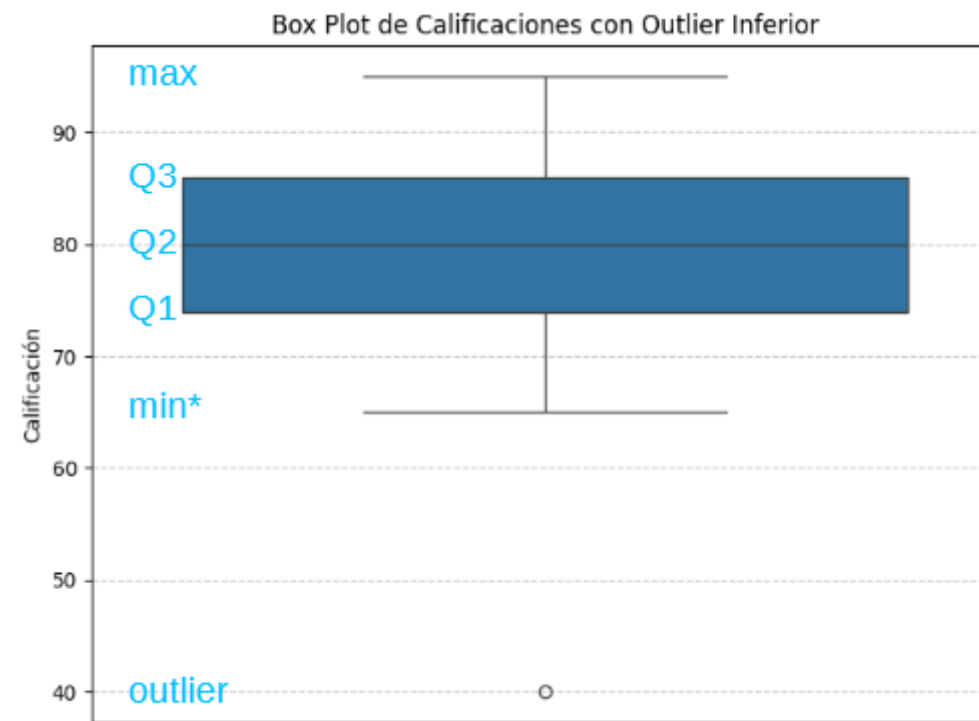
IQR: 12.00

Límite inferior para outliers: 56.00

Límite superior para outliers: 104.00

Valores considerados outliers según la regla del $1.5 * IQR$:

Calificación
40



min*: el mínimo sin considerar el outlier.



Recomendación sobre los outliers:

Se queda: Si el dato es real, aunque sea raro (ej. la fortuna de Bill Gates en un estudio de riqueza).

Se va: Si el dato es un error de medición, un error de carga o no pertenece al grupo de estudio.

Antes de borrar un outlier, lo ideal es documentarlo. En lugar de simplemente borrar la fila, marcarla como "inválida" para que otros sepan por qué no se incluyó en el cálculo final.



Hay 100 percentiles ($P_1, P_2, \dots, P_{100}$).

El k-ésimo percentil (P_k) es el valor por debajo del cual se encuentra el k% de las observaciones de los datos ordenados.

Ejemplo:

Si un estudiante obtiene una calificación en el percentil 95 en un examen → obtuvo una calificación mejor que el 95% de los demás estudiantes.

Dada esta lista: [60, 65, 70, 72, 75, 80, 85, 88, 90, 92, 95] → la lista debe estar ordenada

$$L = \frac{k}{100} \times N$$

N: Número de elementos → 11

k: percentil → 70

L: elemento de la lista que cumple con k.

$L = (70/100) \times 11 = 7.7 \rightarrow 8 \rightarrow$ el elemento 8 (88) es percentil 70.



Actividad

Vamos a seguir el notebook de ejemplo y realizar un ejercicio con un data set de nuestro interes



Actividad

Vamos a seguir el notebook de ejemplo y realizar un ejercicio con un data set de nuestro interes

